Universite Paris Est Creteil
IUT Senart Fontainebleau

SAE 4.1 Chuzzle

Rapport d'avancement



Algassimou Pellel DIALLO

Ayoub ANHDIRE

Youness BOULALAMME

Table des matières

1 Introduction **2**

2 Fonctionnalités **2**
2.1 Fonctionnalités implémentées 2
2.2 Captures d'écran 2

3 Structure du programme **2**
3.1 Modèle 2
3.2 Vue 3
3.3 Contrôleur 3
3.4 Rendu graphique 3
3.5 Diagramme UML 3

4 Algorithme de fin de partie **3**
4.1 Principe 3
4.2 Étapes 3
4.3 Implémentation 3

5 Conclusion **4**
5.1 Auteur 1 4
5.2 Auteur 2 4
5.3 Auteur 3 4

1 Introduction

Dans le cadre de la SAE 4.1, nous avons développé une application Android inspirée du jeu **Chuzzle**.

Le jeu se base sur une grille de taille 6x6 contenant différents éléments colorés. Le joueur peut effectuer des glissements de lignes ou de colonnes afin de créer des séries d'au moins trois éléments identiques.

Lorsqu'une série est formée :

— elle est supprimée,
— les éléments au-dessus tombent (gravité),
— de nouveaux éléments apparaissent.

L'objectif est d'obtenir le meilleur score possible avant qu'aucun coup ne soit possible.

Le projet est structuré selon une architecture **MVC (Modèle - Vue - Contrôleur)**.

2 Fonctionnalités

2.1 Fonctionnalités implémentées

— Génération d'une grille sans série initiale
— Détection des séries
— Suppression des séries
— Application de la gravité
— Remplissage des cases vides
— Calcul du score
— Menu principal
— Lancement d'une partie
— Affichage de la grille
— Détection de fin de partie

2.2 Captures d'écran

FIGURE 1 – Menu principal

3 Structure du programme

3.1 Modèle

— Grille
— Case
— TypeElement
— Serie
— Position
— MoteurJeu
— ScoreManager

3.2 Vue

— MenuActivity
— JeuActivity
— OptionsActivity
— CaseView

3.3 Contrôleur

— MenuController
— JeuController

3.4 Rendu graphique

— ElementRenderer
— NormalRenderer
— DaltonienRenderer

3.5 Diagramme UML

FIGURE 2 – Diagramme de classes

4 Algorithme de fin de partie

La partie se termine lorsqu'aucun coup n'est possible.

4.1 Principe

L'algorithme consiste à tester tous les déplacements possibles sur la grille.

4.2 Étapes

1. Parcourir toutes les lignes
2. Tester les décalages possibles
3. Vérifier si une série est créée
4. Répéter pour les colonnes
5. Si aucun coup ne crée de série :

La partie est terminée

4.3 Implémentation

Cette vérification est réalisée dans la méthode :

`Grille.existeCoupPossible()`

Puis utilisée dans :

`MoteurJeu.jouerCoup()`

5 Conclusion

5.1 Auteur 1

Ce projet m'a permis de comprendre :
— l'architecture MVC
— la logique d'un moteur de jeu
— le développement Android
Difficultés rencontrées :
— gestion des interactions
— affichage graphique
Améliorations possibles :
— animations
— optimisation graphique

5.2 Auteur 2

À compléter.

5.3 Auteur 3

À compléter.